MILVUS ENGINEERING PROTOTYPE · 31 OCTOBER 2026

MultiSearch
Independent searches. One RPC.

A working Proxy implementation, a rebuilt server binary, real client calls, and measured comparisons with ordinary Search.

Standalone verified **Kubernetes verified**

The idea is feasible at the Proxy boundary

The implemented API accepts a bag of complete, independent Search requests. Proxy applies the normal Search entry checks to each item, invokes its ordinary execution path, and returns separate results. The prototype successfully searched collections with different schemas, dimensions, metrics and topK values in one external RPC.

50 → 1
external RPCs per retrieval group
Measured by the client interceptor

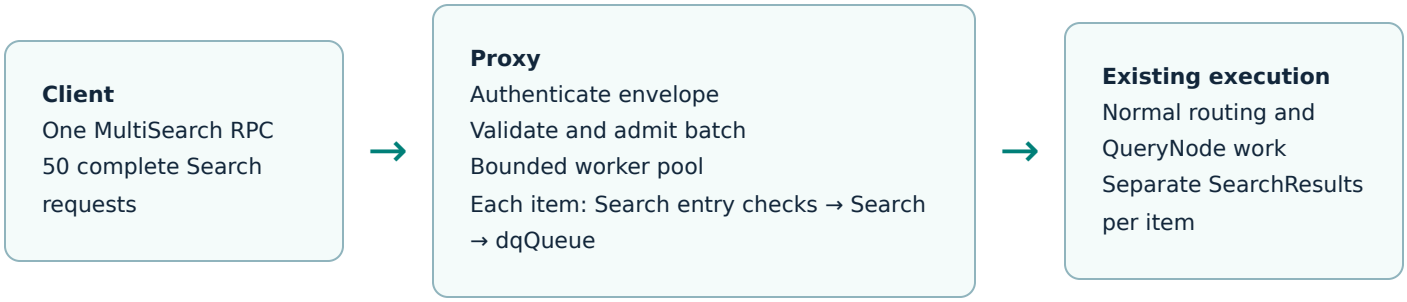
24
heterogeneous collections
6,144 populated rows per environment

2.78×
lower median group latency
Local microbenchmark, concurrency 1

This directly resolves the schema/metric objection in the discussion: a unified cross-collection ranking would require compatible scores and a defined merge operation. MultiSearch does neither. Each item validates its own collection and performs its own normal shard-level reduction. There is no reduce across items and no atomic cross-collection snapshot.

> **Evidence supports proceeding to API design review.** The implementation and end-to-end tests demonstrate that this orchestration can live in Proxy without changes to QueryNode, Segcore, indexes or WAL behavior. Production readiness, customer latency targets and PSC incident mitigation require additional validation.

Contract and execution path



The fan-out happens in process, into the existing Proxy Search entry path. No loopback network calls are introduced. The dispatcher has no knowledge of schemas, shards, indexes, or score merging. Each item keeps its own nq and quota accounting; packing collections together does not turn them into one larger-nq collection search. Vector and result payloads still have to cross the network.

Decision	Prototype behavior
API name	MultiSearch; Python adapter method multi_search(...)
Wire entry	/milvus.proto.multisearch.MultiSearchService/MultiSearch on the existing Proxy gRPC port
Request	One database plus ordered {id, SearchRequest} items; repeated collections allowed
Independence	Different fields, vector dimensions, metrics, filters, topK and output fields per item
Result	Ordered {id, SearchResults} or per-item gRPC code/message; envelope success does not mean all items succeeded
Partial failure	Ordinary item failures preserve successful siblings; no automatic retries
Whole-envelope failure	Invalid envelope, request/response size, admission rejection, or overall deadline; no partial-results recovery on a lost unary response
Database	Envelope DB → connection metadata → default. Conflicting item DBs rejected before dispatch
Consistency	Each item retains ordinary Search consistency. No cross-item snapshot or atomicity

Entry checks must run per item

Calling Proxy.Search directly would bypass network interceptors. This implementation refactors the existing Search entry chain into a shared function, then explicitly runs it for every item using Search's real method name and request type. That preserves database handling, collection RBAC, per-item quotas, hooks, access logging and ordinary standalone forwarding. The envelope has its own explicit authentication/access chain; it does not rely on permissive fallbacks for unknown request types.

Requests are cloned before dispatch. Mutable RPC statistics and access-log state are isolated per item; identity, deadline and trace ancestry are inherited. Cloud-only gateway and billing hooks are not available in this OSS environment and remain a release gate.

Guardrail	Default	Scope
Batch size	128 items	Validated before dispatch
Request size	4 MiB	Serialized envelope
Per-batch concurrency	16	Independent Search callbacks
Global concurrency	64	MultiSearch items per Proxy; ordinary Search keeps existing controls
Active envelopes	8	Fail-fast admission, no unbounded waiting batches
Response budget	16 MiB	Retained serialized outcomes; not a bound on all temporary native allocations
Maximum duration	30 seconds	Earlier client deadline wins

What batching changes about failures

Under the illustrative assumption of independent 3% failures per external RPC, the chance of at least one failure in a group is $1 - (1 - 0.03)^n$: 78.2% for 50 RPCs and 95.2% for 100. A single envelope has 3% exposure in that simplified model; two envelopes have 5.91%, and three have 8.73%.

These are model calculations, not measured PSC behavior. Real network failures may be correlated, and larger payloads or longer requests may change failure probability. A failed envelope affects all its items. Adaptive model search rounds still need separate calls. Internal search failures and total database work do not disappear.

Returning partial outcomes does not relax the application’s requirement that every required search succeed. It makes the failed subset visible. A client can later retry only failed items when it receives a usable response, subject to an overall deadline, backoff and a single retry owner. Losing the entire response can require replaying the bag and repeating successful work. This prototype deliberately adds no retry policy.

The code that demonstrates the design

These excerpts are extracted from the implemented files. The complete protocol and dispatcher are small enough to review directly; generated protobuf bindings and the existing Search engine provide the rest.

pkg/proto/multisearch.proto · actual prototype source

```
syntax = "proto3";
package milvus.proto.multisearch;

option go_package = "github.com/milvus-io/milvus/pkg/v3/proto/multisearchpb";
```

```
import "common.proto";
import "milvus.proto";

// Prototype extension on the public Proxy gRPC port. Items are independent
// ordinary searches, not a cross-collection ranking or snapshot operation.
service MultiSearchService {
  rpc MultiSearch(MultiSearchRequest) returns (MultiSearchResponse) {}
}

message SearchItem {
  string id = 1;
  milvus.SearchRequest search = 2;
}

message MultiSearchRequest {
  string db_name = 1;
  repeated SearchItem items = 2;
}

message SearchOutcome {
  string id = 1;
  // Present for ordinary Milvus outcomes, including non-success Status.
  milvus.SearchResults result = 2;
  // Transport errors from the ordinary Search entry chain remain distinct
  // from Milvus Status. Zero means OK; nonzero means result is absent.
  int32 grpc_code = 3;
  string grpc_message = 4;
}

message MultiSearchResponse {
  // Envelope status only: success does not imply every item succeeded.
  common.Status status = 1;
  // Same order and cardinality as input when envelope status is successful.
  repeated SearchOutcome items = 2;
}
```

Re-enter ordinary Search at the correct boundary

The callback uses the same entry chain as network Search calls, then calls Server.Search. This is the key difference between a safe orchestration primitive and an internal loop that skips authorization or quotas.

internal/distributed/proxy/multisearch.go · actual prototype source

```
    entry := proxyEntryInterceptor(limiter)
    search := func(ctx context.Context, req *milvuspb.SearchRequest) (*milvuspb.SearchResults, error) {
        // Fork mutable stats; the access-log interceptor similarly allocates
        // an independent object. Identity, deadlines and tracing are inherited.
        ctx = metrics.WithIndependentRPCStats(ctx)
        resp, err := entry(ctx, req, &grpc.UnaryServerInfo{Server: s, FullMethod:
milvuspb.MilvusService_Search_FullMethodName},
            func(ctx context.Context, req any) (any, error) {
                return s.Search(ctx, req.(*milvuspb.SearchRequest))
            })
        if err != nil {
            return nil, err
        }
        result, ok := resp.(*milvuspb.SearchResults)
        if !ok || result == nil {
            return &milvuspb.SearchResults{Status: merr.Status(merr.WrapErrServiceInternalMsg("Search entry returned an
```

```
invalid response"))}, nil
        }
        return result, nil
    }
```

Bounded workers, isolated requests, independent outcomes

internal/proxy/multisearch.go · actual prototype source

```
    for range min(e.limits.Parallelism, len(req.Items)) {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for ctx.Err() == nil {
                i := int(next.Add(1) - 1)
                if i >= len(req.Items) {
                    return
                }
                select {
                case e.active <- struct{}{}:
                case <-ctx.Done():
                    return
                }
                if ctx.Err() != nil {
                    <-e.active
                    return
                }
                item := req.Items[i]
                // Search and entry interceptors may mutate the request. Never share
                // caller-owned request pointers between workers (or retry attempts).
                search := proto.Clone(item.Search).(*milvuspb.SearchRequest)
                search.DbName = db
                result, err := e.search(ctx, search)
                <-e.active
                outcome := &multisearchpb.SearchOutcome{Id: item.Id, Result: result}
                if err != nil {
                    s := status.Convert(err)
                    outcome.Result = nil
                    outcome.GrpcCode, outcome.GrpcMessage = int32(s.Code()), s.Message()
                } else if result == nil || result.Status == nil {
                    outcome.Result = &milvuspb.SearchResults{Status: merr.Status(merr.WrapErrServiceInternalMsg("Search
returned no result status"))}
                }
                // Reserve protobuf framing overhead as well as outcome bytes.
                if resultBytes.Add(int64(proto.Size(outcome)+16)) > int64(e.limits.MaxResponseBytes) {
                    overflow.Store(true)
                    cancel()
                    return
                }
                out.Items[i] = outcome
            }
        }()
    }
```

The client really sends one request

The local adapter reuses PyMilvus's existing Search messages and adds one extension service stub. This is a functioning prototype method, not a claim that released MilvusClient already supports MultiSearch.

examples/multisearch/client.py · actual prototype source

```
def multi_search(self, items, timeout=10):
    """Return all outcomes, including individual failures, without retrying."""
    request = multisearch_pb2.MultiSearchRequest(db_name=self.db_name)
    for item_id, search_request in items:
        request.items.add(id=item_id, search=search_request)
    return self._multi(request, timeout=timeout, metadata=self.metadata)
```

Public protocol ownership is intentionally separated for this prototype: the extension lives in the Milvus repository and imports the existing public Search messages. Graduation requires an agreed public protobuf/API location, SDK support and compatibility behavior.

Functional validation before performance

The final binary ran in a standalone Docker container with authentication enabled. The final standalone verification reused the deterministic fixture populated by the Python harness: 24 collections × 256 rows, vector dimensions 8/16/32, metrics L2/IP/COSINE, differing field names, integer/string primary keys, FLAT indexes and strong consistency. A 50-item MultiSearch was compared with 50 ordinary Search calls by IDs, per-query topKs, scalar output fields and scores (tolerance 1e−5).

Verification	Result	Evidence
Populated heterogeneous collections	PASS	24 collections, 6144 rows; mixed fields, dimensions, metrics and primary-key types
50-item equivalence and one RPC	PASS	50 independent searches across 24 collections; IDs, topks, scalar fields and scores match ordinary Search
Partial failure isolation	PASS	[{"id": "valid-a", "code": 0, "grpc_code": 0, "reason": ""}, {"id": "missing", "code": 100, "grpc_code": 0, "reason": "collection not found[database=default][collection=ms_prototype_missing]"}, {"id": "bad-dim", "code": 2000, "grpc_code": 0, "reason": "failed to search: failed to search/query delegator 2 for channel by-dev-rootcoord-dml_1_470146438620250081v0: fail to search on QueryNode 2: worker(2) query failed: parser searchRequest failed: => vector dimension mismatch, expected vector size(byte) 64, actual 8. at ../internal/core/src/query/Plan.cpp:183: segcore error[segcoreCode=2001]"}, {"id": "valid-b", "code": 0, "grpc_code": 0, "reason": ""}, {"id": "empty", "code": 0, "grpc_code": 0, "reason": ""}]
Envelope rejects empty	PASS	{"code": 1100, "reason": "MultiSearch requires 1..128 items: invalid parameter"}

Verification	Result	Evidence
Envelope rejects duplicate IDs	PASS	{"code": 1100, "reason": "duplicate MultiSearch item id \"x\": invalid parameter"}
Envelope rejects 129 items	PASS	{"code": 1100, "reason": "MultiSearch requires 1..128 items: invalid parameter"}
Per-item collection authorization	PASS	{"allowed": 0, "denied_grpc_code": 7}
Envelope authentication	PASS	UNAUTHENTICATED
Client deadline	PASS	DEADLINE_EXCEEDED
Search succeeds after deadline	PASS	server continues serving requests

Additional real-server boundary and quota checks

Verification	Result	Evidence
Request byte limit	PASS	{"code": 1102, "bytes": 4194509}
Single-database envelope	PASS	{"code": 1100}
Real collection quota isolation	PASS	{"ordinary_code": 9, "multisearch_item_code": 9, "other_item_code": 0, "retriable": false}

The quota test temporarily set one collection's search quota to zero, observed the same status from ordinary Search and the corresponding bag item, and verified that a permitted sibling succeeded. The quota was restored before the final benchmark. This exercises the real rate-limit path, not a mocked callback.

Failure-path audit and verification scope

Failure / invariant	Evidence	Boundary of the claim
Missing collection	Live Search and MultiSearch return code 100	Existing lookup/status preserved
Wrong vector dimension	Live result code 2000; reason includes Segcore code 2001	Existing upstream classification is preserved, not corrected

Failure / invariant	Evidence	Boundary of the claim
Collection permission denied	Live per-item gRPC code 7; allowed sibling succeeds	Public OSS RBAC exercised
Invalid credentials	Live envelope gRPC code 16	Authentication before dispatch
Deadline	Live client code 4 followed by a successful request; unit tests stop queued dispatch	No guarantee of immediate termination of all native work
Quota zero	Live code 9, retriable=false, matching ordinary Search	Zero-quota denial tested; every other overload origin is not fault-injected
Admission / concurrency	Focused unit and race tests	Limits and slot release tested with controlled callbacks
Response overflow / nil result	Focused unit tests	Not a live-server memory exhaustion test
Mutable statistics	Concurrent race test	Independent request state verified
Existing Proxy service	Test_NewServer regression subtests pass with disposable etcd	Focused regression, not full repository test suite

The relevant status paths were inspected at their origins and consumers: metadata lookup, Segcore dimension validation, authentication/privilege interception, quota denial and deadline propagation. No generic error reclassification or retry behavior was added. Object-storage throttling, corruption, OOM, node crashes, PSC jitter and lost responses were not injected. Their recovery behavior is not claimed.

▸ **Test execution notes and discarded runs**

Performance against ordinary Search

Each measured group performs the same 50 logical searches. Baselines use a persistent gRPC channel: sequential Search for reference and bounded client fan-out as the useful comparison. Parallel Search and MultiSearch both allow 16 searches per group and 64 active searches globally. There are no retries. Warmup is excluded; every returned item is checked for success.

Standalone Docker was limited to 16 CPUs and 32 GiB on a 128-logical-CPU host. Measurements were taken before the Kubernetes workload was deployed. Client CPU is Python process CPU; server CPU is the standalone container's cgroup CPU delta, including its background activity.

Mode	Concurrent groups	Groups	RPCs	p50 ms	p95 ms	p99 ms	Logical searches/s	Errors
Sequential Search	1	20	1000	332.98	341.97	349.61	150	0

Mode	Concurrent groups	Groups	RPCs	p50 ms	p95 ms	p99 ms	Logical searches/s	Errors
Parallel Search	1	60	3000	60.25	63.75	81.27	822	0
MultiSearch	1	60	60	21.67	24.66	30.77	2,263	0
MultiSearch	4	60	60	27.92	44.66	45.06	6,544	0
Parallel Search	4	60	3000	238.26	261.01	284.63	840	0
Parallel Search	8	60	3000	517.14	663.54	718.19	776	0
MultiSearch	8	60	60	56.08	79.39	79.89	6,776	0

Mode	Concurrent groups	Measured wall s	Client CPU s / 1k searches	Server CPU s / 1k searches
Sequential Search	1	6.672	1.238	7.893
Parallel Search	1	3.648	1.561	6.150
MultiSearch	1	1.326	0.057	4.166
MultiSearch	4	0.458	0.048	2.415
Parallel Search	4	3.571	1.553	6.102
Parallel Search	8	3.868	1.692	6.289
MultiSearch	8	0.443	0.049	2.354

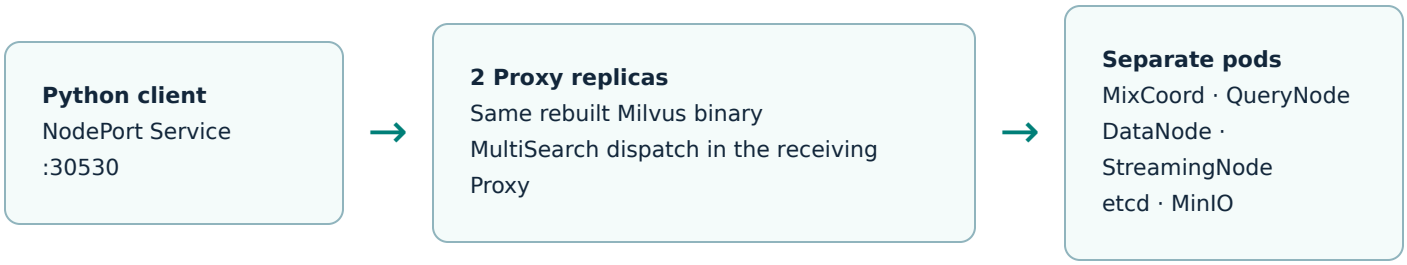
Observed at concurrency 1: median completion time fell from 60.25 ms to 21.67 ms (64.0% lower), with 2.75× the completed logical-search throughput. Client CPU per logical search fell by 96.4%. External RPC count fell 50×; the number of logical Search operations did not.

Interpret this as an orchestration microbenchmark. The dataset is small, searches are cheap, and Python client overhead is prominent. Each parallel/batch phase contains only 60 groups; high-concurrency MultiSearch phases last under one second. The p99 values are descriptive sample percentiles, not stable tail estimates. The baseline is this Python gRPC client, not the customer's Baseten C runtime / REST client. This is neither sustained saturation capacity nor evidence of production speedup, server efficiency at scale, or a PSC reliability fix.

All seven final phases completed: 19,000 measured logical searches, zero observed errors. The likely mechanism is reduced Python/gRPC work and moving fan-out into the Go Proxy; the experiment does not isolate each cost with profiling. Larger vectors, ANN indexes, filters, larger outputs, heterogeneous slow items and a remote network should be tested next. A unary envelope still waits for all items, so a slow item controls completion latency.

Distributed Kubernetes functional validation

PASS The same image ran in an isolated Minikube v1.30.0 cluster (12 CPUs, 24 GiB node budget). The client connected through a Kubernetes NodePort Service at `192.168.58.2:30530`, not to the standalone Docker endpoint.



Verification	Result	Evidence
Populated heterogeneous collections	PASS	24 collections, 6144 rows; mixed fields, dimensions, metrics and primary-key types
50-item equivalence and one RPC	PASS	50 independent searches across 24 collections; IDs, topks, scalar fields and scores match ordinary Search
Partial failure isolation	PASS	[{"id": "valid-a", "code": 0, "grpc_code": 0, "reason": ""}, {"id": "missing", "code": 100, "grpc_code": 0, "reason": "collection not found[database=default][collection=ms_prototype_missing]"}, {"id": "bad-dim", "code": 2000, "grpc_code": 0, "reason": "failed to search: failed to search/query delegator 4 for channel by-dev-rootcoord-dml_1_470146758697025608v0: fail to search on QueryNode 4: worker(2) query failed: parser searchRequest failed: => vector dimension mismatch, expected vector size(byte) 64, actual 8. at ../internal/core/src/query/Plan.cpp:183: segcore error[segcoreCode=2001]"}, {"id": "valid-b", "code": 0, "grpc_code": 0, "reason": ""}, {"id": "empty", "code": 0, "grpc_code": 0, "reason": ""}]
Envelope rejects empty	PASS	{"code": 1100, "reason": "MultiSearch requires 1..128 items: invalid parameter"}
Envelope rejects duplicate IDs	PASS	{"code": 1100, "reason": "duplicate MultiSearch item id \"x\": invalid parameter"}
Envelope rejects 129	PASS	{"code": 1100, "reason": "MultiSearch requires 1..128 items: invalid parameter"}

Verification	Result	Evidence
items		
Per-item collection authorization	PASS	{"allowed": 0, "denied_grpc_code": 7}
Envelope authentication	PASS	UNAUTHENTICATED
Client deadline	PASS	DEADLINE_EXCEEDED
Search succeeds after deadline	PASS	server continues serving requests

This verifies that the extension can use the ordinary distributed execution path across component pods. It does not establish multi-node resilience, rolling-upgrade compatibility, or Kubernetes performance. The cluster has one local Kubernetes node and ephemeral volumes. StreamingNode also embeds its ordinary QueryNode functionality.

Both Proxy replicas called directly

Proxy pod	Items / RPCs	Binary SHA-256	Result
proxy-7c6d54599b-bfdb8	50 / 1	bbadbcbe91f24648ee862f8aa83807f7601b46be3b81f6570cb30d864ab62afe	PASS
proxy-7c6d54599b-r6zz8	50 / 1	bbadbcbe91f24648ee862f8aa83807f7601b46be3b81f6570cb30d864ab62afe	PASS

Temporary localhost port-forwards targeted each Pod individually. Each returned 50 successful item outcomes in one RPC; both binary checksums matched the standalone build. See per-Proxy evidence.

During cluster setup, kube-proxy could not start because the host limit of 128 inotify instances was exhausted. Raising `fs.inotify.max_user_instances` to 1024 and restarting the affected test-cluster system pods restored networking. This was an environment setup issue, not a change to MultiSearch. The setting remains raised while the local cluster is running. An initial deployment also exposed Kubernetes service-link environment variables overriding Milvus's MINIO_PORT setting; the final manifest uses enableServiceLinks: false. Dependencies use a Recreate deployment strategy for this ephemeral test cluster.

Engineering work before production

Area	Prototype evidence	Remaining work
Core feasibility	Real heterogeneous searches work through existing Proxy execution	Review API naming/ownership and upstream implementation
SDK / compatibility	One-call Python adapter works	Official SDK ergonomics, TLS, public proto rollout, unsupported-server detection, full gRPC error details
Authorization / quotas	Live OSS authentication, collection RBAC and quota isolation pass	Managed gateway allowlists, private auth/billing hooks and deployed Cloud integration
Resource controls	Bounded batches/workers/bytes and tested cancellation	Tenant fairness, sustained mixed workloads, memory profiles and cancellation cleanup
Observability	Per-item existing Search hooks/metrics/access path reused; stats isolated	Envelope transport-byte accounting, dedicated batch metrics and tracing review
Performance	Lower overhead in measured local microbenchmark	Real customer payloads, index sizes, remote links, long runs and repeated randomized phases
Reliability	Independent item errors preserved	Correlated PSC fault injection, deadline-aware retry owner, lost-response recovery policy
Rollout	Same binary works locally in two deployment modes	Mixed-version clusters, canary rollout and production resource defaults

The proposed next step is an API/Proxy design review backed by this runnable prototype. There is evidence for technical feasibility and local overhead reduction. There is not yet evidence to promise a specific customer SLO or to remove the customer's framework without validating all of its responsibilities.

Build provenance and reproducible evidence

Artifact	Identity
Source base commit	d49b1eb6c599295cfc1f4e918d171c582243088e
Go toolchain	go version go1.26.5 linux/amd64
Binary	bin/milvus — rebuilt Go server with -tags dynamic

Artifact	Identity
Binary SHA-256	bbadbcbe91f24648ee862f8aa83807f7601b46be3b81f6570cb30d864ab62afe
Container binary SHA-256	bbadbcbe91f24648ee862f8aa83807f7601b46be3b81f6570cb30d864ab62afe
Docker image	sha256:be80787c0d446b51eae191f6c2ae73306d0c4b54f7af1b76217799ceab0158c7
Native runtime	Unmodified shared libraries from cached milvusdb/milvus:v3.0.0; Go binary rebuilt with -tags dynamic

The Go binary was rebuilt from the modified source. Unchanged native C++/Rust shared libraries were reused from the pinned Milvus v3.0.0 runtime image; a full native rebuild was not performed. The container's binary checksum matches the host artifact. Source changes, build scripts, test fixtures and the deployment manifest are supplied for engineering review.

```
# From the repository root
bash examples/multisearch/build_prototype.sh
bash examples/multisearch/run_prototype.sh
.venv-multisearch/bin/python examples/multisearch/verify.py --functional-only
.venv-multisearch/bin/python examples/multisearch/verify_limits.py
.venv-multisearch/bin/python examples/multisearch/benchmark.py
# Kubernetes commands, dependencies and focused Go checks are in the README.
python3 examples/multisearch/render_report.py
```

The build helper assembles the component commands used in this experiment. A fresh-machine end-to-end run of that combined helper remains unverified. Exact commands, prerequisites, safe fixture scope and cleanup instructions are in the prototype README.

- Design document · Build and run instructions · Kubernetes deployment
- Standalone functional results · Boundary and quota results · Performance raw samples · Kubernetes functional results
- Build manifest · Verification scope · Final standalone log · Dispatcher race tests · Proxy regression log
- Dispatcher · Search entry adapter · Protocol · Client

The report is self-contained for reading and printing. Evidence links are relative to the repository/artifact bundle. Private kubeconfig credentials are excluded.